gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

DAT321 DIT843
Basics in Software Testing

Dr. Francisco Gomes de Oliveira Neto (he / him)

Elaine Weyuker

- Harlan D. Mills Award (2004) for **leading research** on software testing and reliability.
- ACM SIGSOFT Outstanding Research Award for deep **and lasting contributions** and impact to software engineering **as a discipline.**

What will we learn today?

- What are the main steps of a **test process**?
- How does the **levels** of testing **relate** to **failures and faults**?
- What is a **test case**? How do we create them?
- What is a **test oracle?** Why do they affect test automation?
- What is **black-** and **white-box** testing? How do we distinguish them?

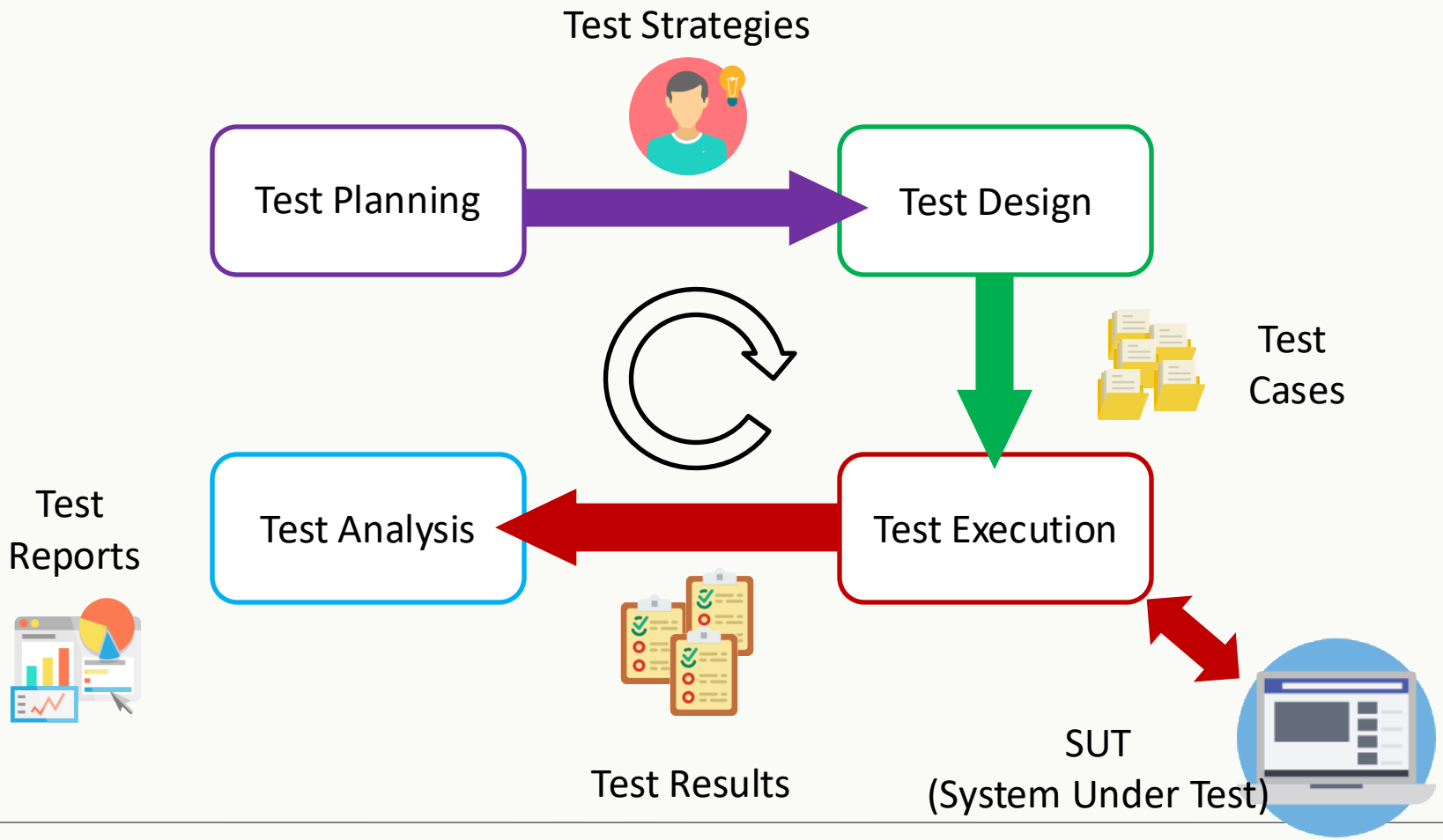
© Francisco Oliveira Neto - 2018

Part 1:

Test as a Process

© Francisco Oliveira Neto - 2018

Test Activities - Overview



© Francisco Oliveira Neto - 2018

Test Activities - Overview

Definition of **test goals, scope**, time constraints, etc.
Can use information from the QA plan.

Test Planning

Test Strategies

Test Design

Creation of test cases to exercise the system behaviour. Mostly done manually but can be automated.

Test Cases

Test Execution

Run the test cases against the System Under Test to compare expected and the actual outputs.

SUT (System Under

Test Results

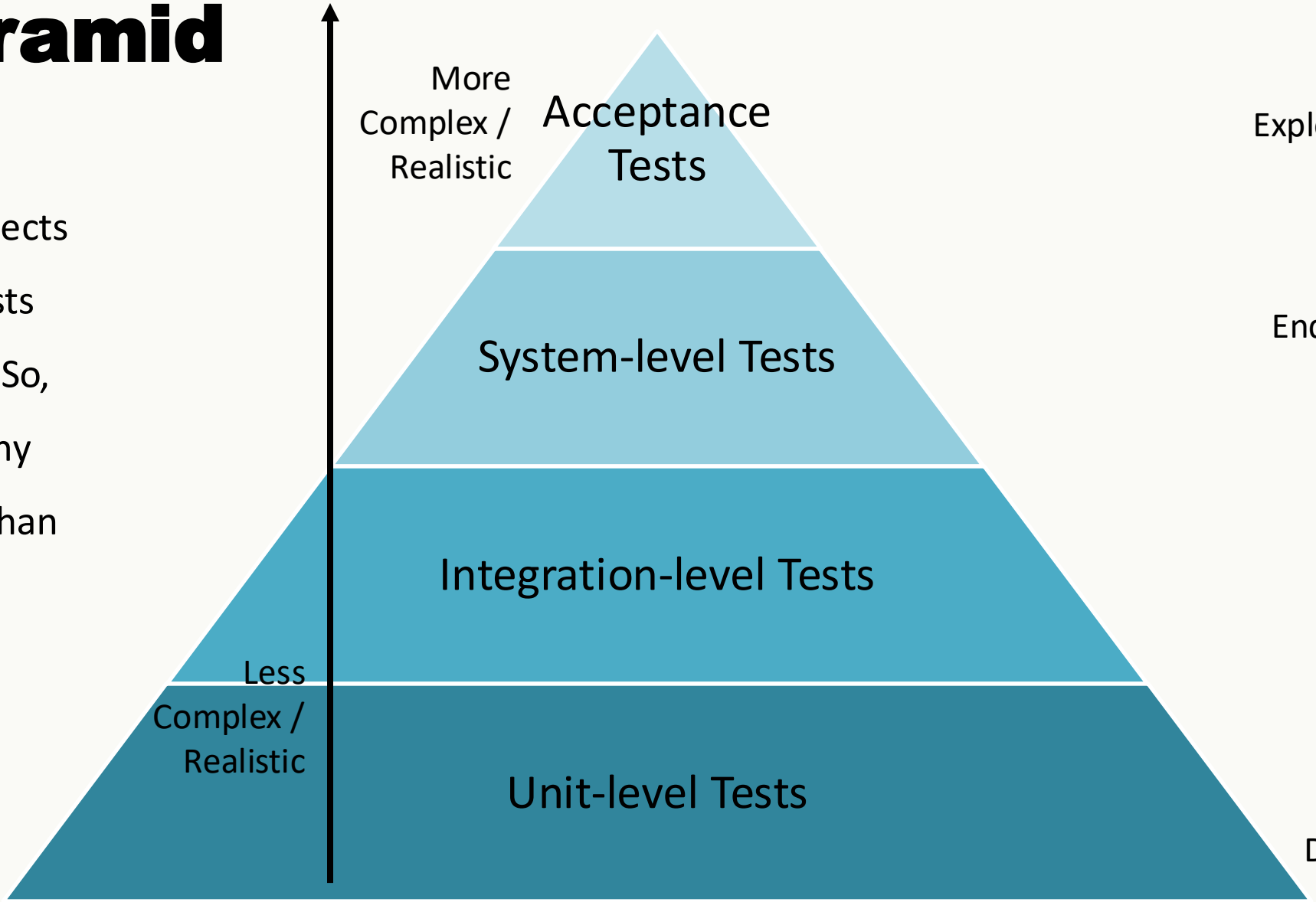
Test Analysis

Read and interpret the results of the tests by evaluating which tests passed and which failed to start the debugging phase.

© Francisco Oliveira Neto - 2018

Test Pyramid

The pyramid shape reflects the **proportion** of tests created at each level. So, we typically find many more unit-level tests than system-level ones



Examples:

Exploratory Tests

End-2-end tests

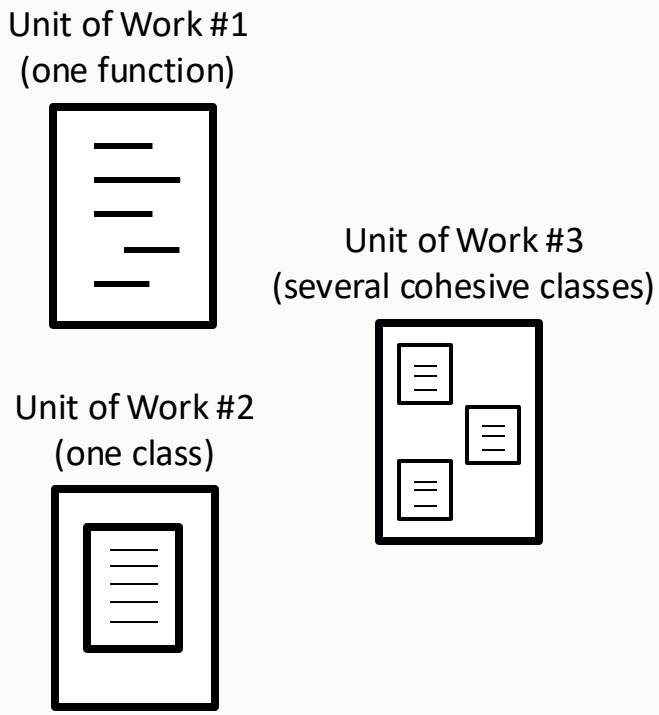
API Testing

Test-Driven Development

© Francisco Oliveira Neto - 2018

Unit Testing

- Piece of code that **invokes** a unit of work to check **one end result.**
- Unit testing is important **because**…
  - Helps developers **understand unit's behavior**;
  - **Automation** of test activities.
  - Executable documentation.



© Francisco Oliveira Neto - 2018

Integration- and System-level Tests

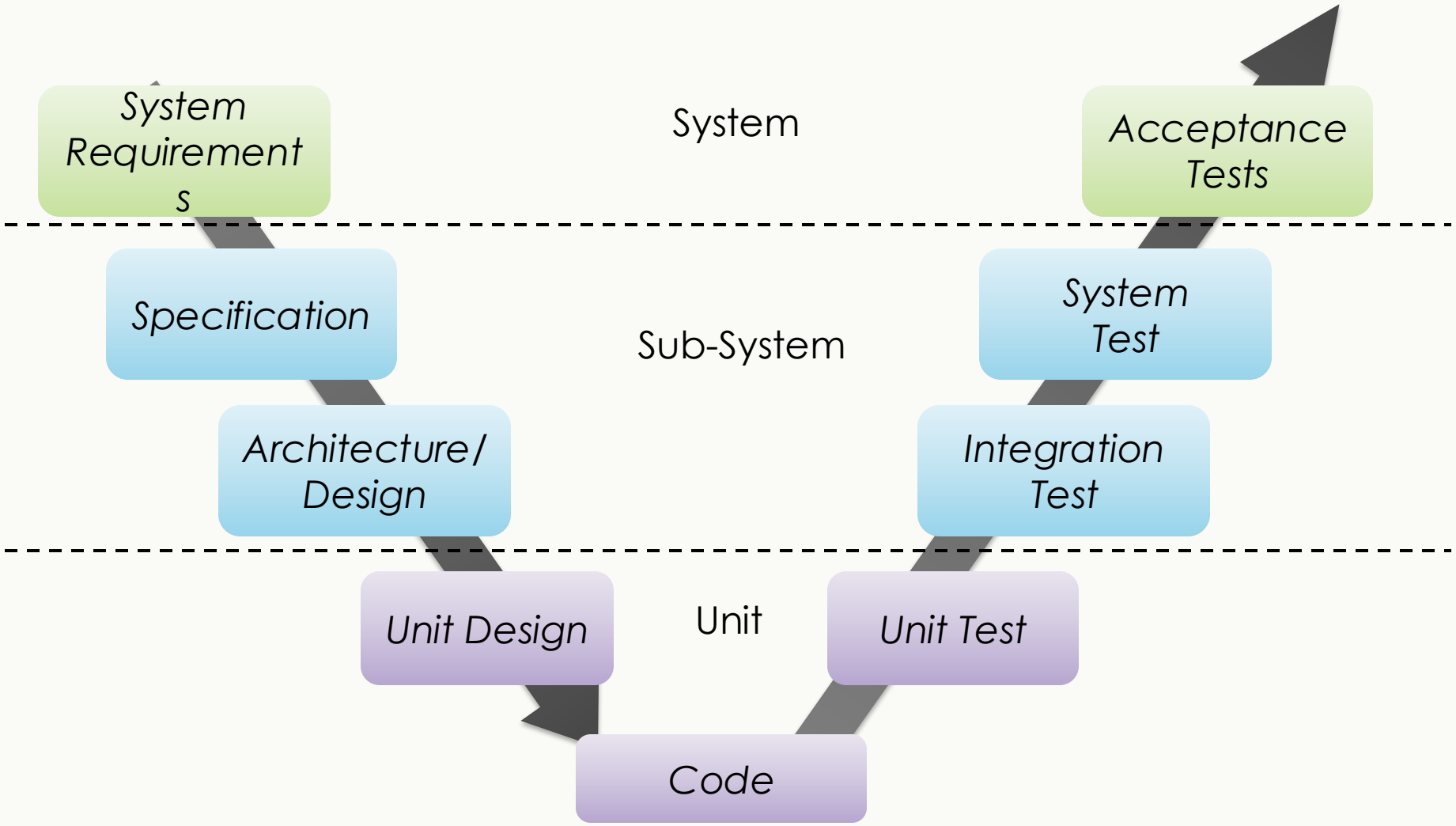
- **Integration-level tests** focus on the **interaction** of different parts, such as their communication, logical dependencies, and exchanged data.
- **System-level tests** focus on exercising **multiple components and the environment** (e.g., operating system, database, external APIs, etc.).

- Drawing a line between those two is hard. Focus on the unique aspects of each test-level to truly understand them.

© Francisco Oliveira Neto - 2018

V model

The V-model is **another representation** to capture the different levels of testing



© Francisco Oliveira Neto - 2017

Manual or Automated Test Execution?

- **Higher** levels of testing are **harder to automate** because the system is composed of varied components written in different programming languages or different software platforms.

- **Manual** test execution is **tedious, error prone** and should be avoided for lower level testing (unit + integration). Can be **useful for system and acceptance** testing such as Exploratory Testing.
- On **unit** level, **automated** test execution is **fast, cheap** and with extensive **tool support** (xUnit test frameworks). Automated tests are at the core of modern development practices such as Continuous Integration.

© Francisco Oliveira Neto - 2018

Part 2:
Test Artefacts

© Francisco Oliveira Neto - 2018

What is a test case?

- **Set of conditions** checking whether the system/application **works as expected**.
- Basic elements:
  - Pre-conditions
  - User actions (Input)
  - **Expected outputs** (post-conditions)

© Francisco Oliveira Neto - 2017

What is a test case?

TC-ID:

Purpose:

Pre-conditions:

There is no available memory in the phone

Steps: *(Copy and paste rows to the table to add more steps, if needed)*

User action	Expected system response
Click contact app	Main Screen is shown
Select 'add new contact'	Contact's fields are displayed
Fill in fields and press 'Done'	Show msg 'Not enough space'

Additional information:

Priority *(Leaving this blank means that the test case priority is none):*

☐ High ☐ Medium ☐ Low ☐ None

** High priority means that the Test Case should be among the first to execute when testing.*

Pre-conditions

Input

Expected Output

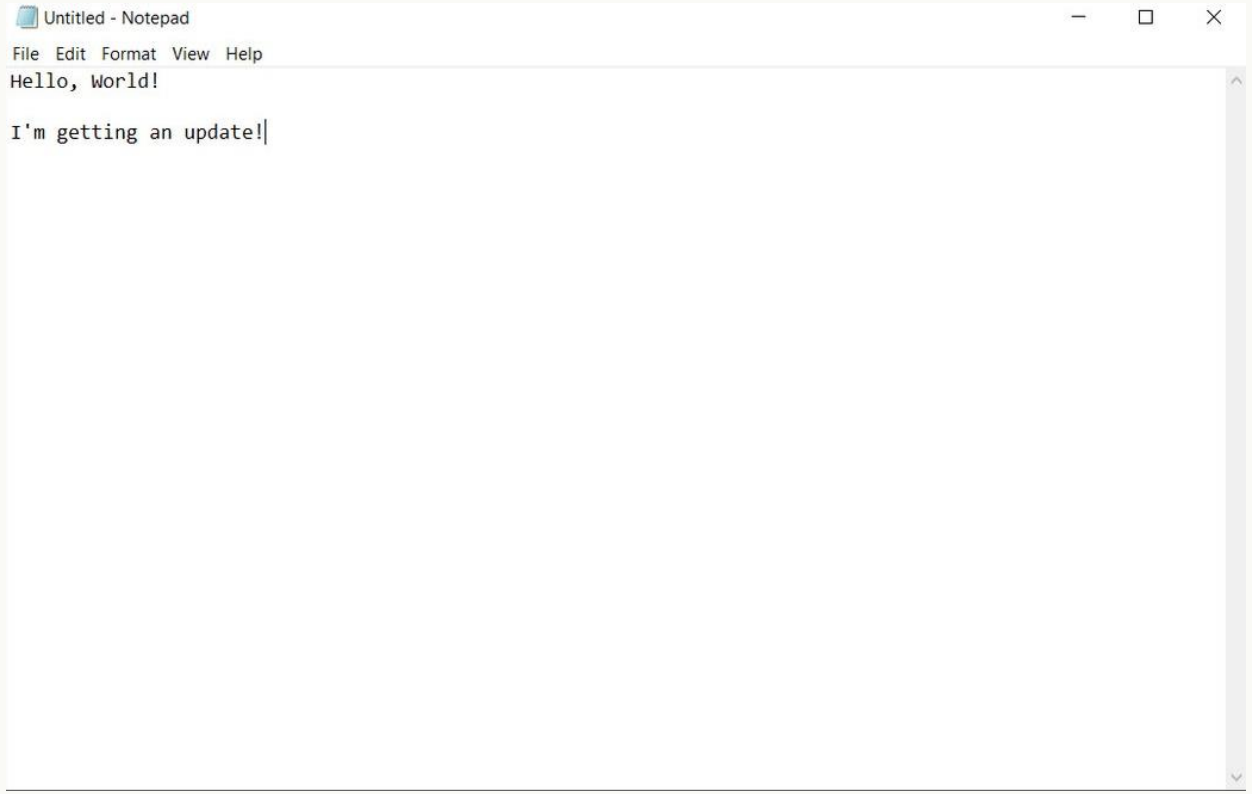
```java
public class Epic2RegularTests {
    private Facade facade;

    @BeforeAll
    public static void systemSetup(){
        // Forces the system config to use "." for decimal separators.
        Locale.setDefault(Locale.US);
    }

    @BeforeEach
    public void testSetup(){
        facade = new Facade();
        //Creates one item and adds it to the database.
        String itemID = "ID1";
        String itemName = "Black T-shirt";
        double price = 150.99;
        facade.createItem(itemID, itemName, price);
    }
    @Test
    public void shouldRemoveItem(){
        // Verifies the attempt to remove a registered item
        String expectedMessage = "Item ID1 was successfully removed.";
        String itemID = "ID1";
        assertEquals(expectedMessage, facade.removeItem(itemID));
        assertFalse(facade.containsItem(itemID));
    }
```


© Francisco Oliveira Neto - 2017

Exercise – Designing Test Cases

- You were hired to test the **Notepad** application. It's a simple word editor.

- Write as many **test cases** as you can for that application.



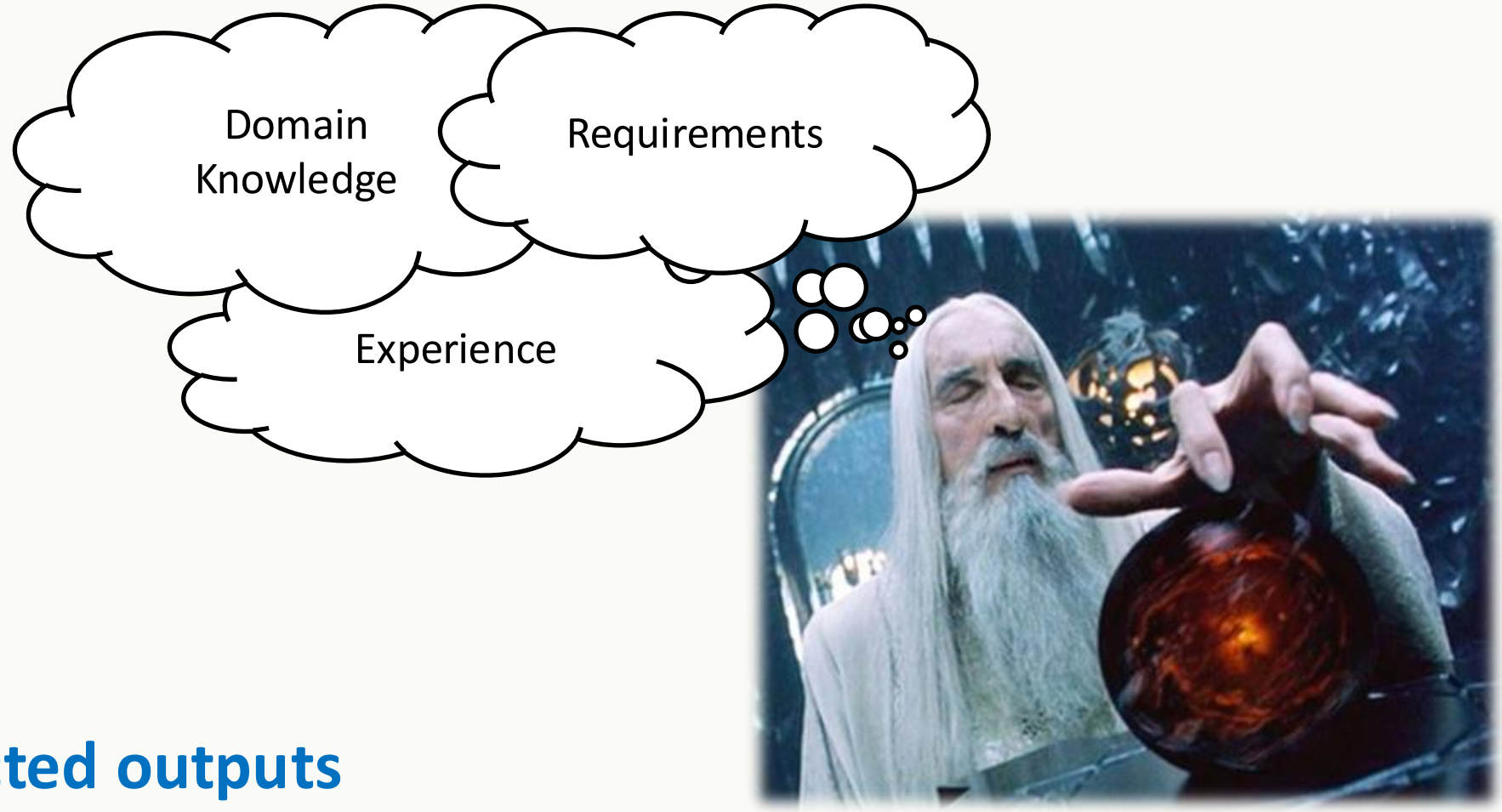
© Francisco Oliveira Neto - 2018

Choosing expected outputs

- Example: You should **write tests** for the function:
  - `absolute_max(a,b)`
  - `Input: a = 5, b = 10. Output:`
  - `Input: a = 0, b = 0 . Output:`
  - `Input: a = -3, b = 1. Output:`

© Francisco Oliveira Neto - 2018

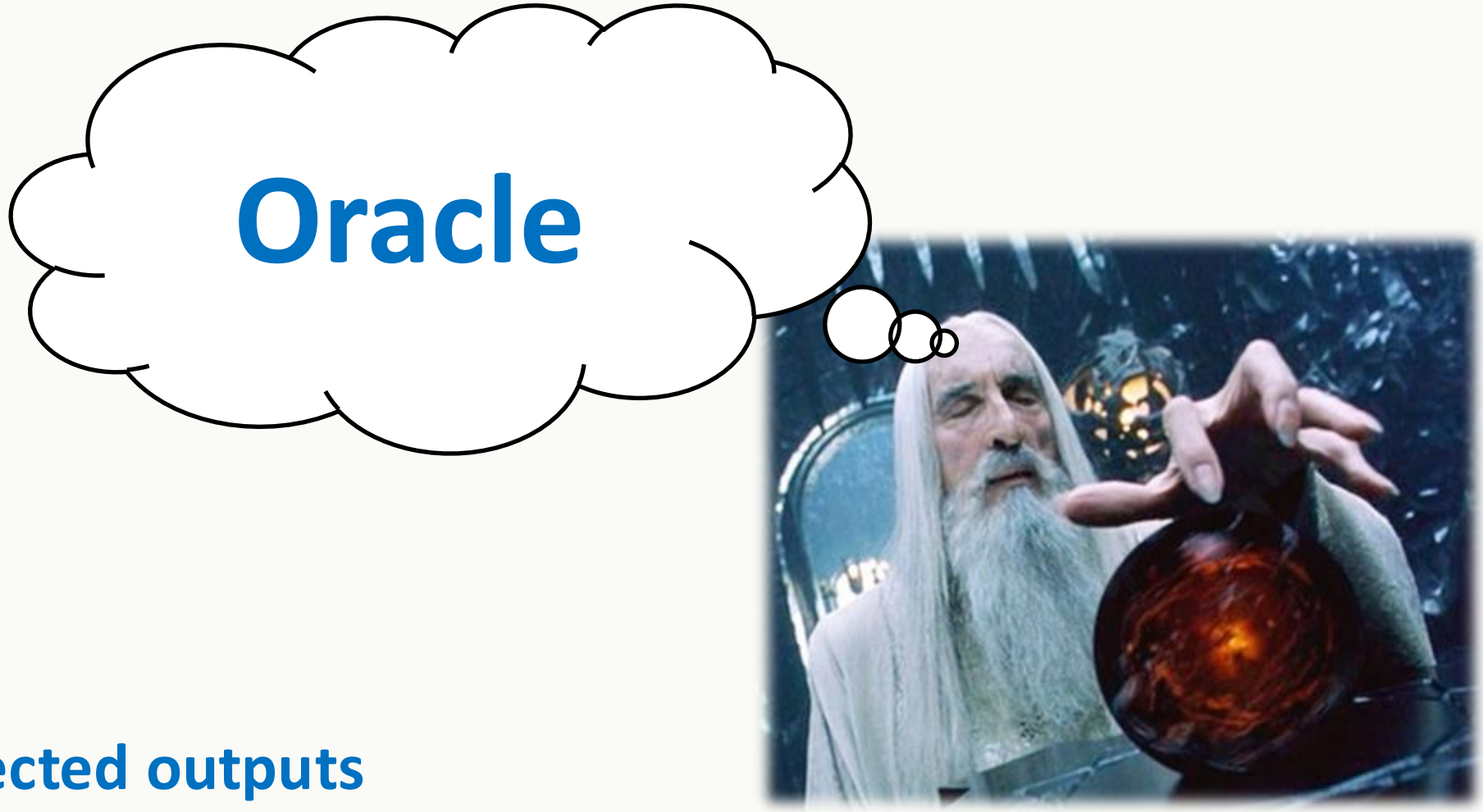
Choosing expected outputs



- **Expected outputs**

© Francisco Oliveira Neto - 2017

Choosing expected outputs



- **Expected outputs**

© Francisco Oliveira Neto - 2017

Test Oracles and Their "Problem"

- Oracles are the **mechanisms** used to determine if the **output / behaviour** of the software under test is correct (**passed**) or not (**failed**).
- Examples:
  - A written software specification is an oracle. **Code should not be an oracle.**
    - Code itself is not an oracle because it doesn't inherently evaluate correctness.
  - The properties of an algorithm can also define an oracle. For instance, the oracle of a sorting algorithm is that the value in position **i** should be less than the value in position **i+1.**

© Francisco Oliveira Neto - 2018

Test Oracles and Their "Problem"

- **"The oracle problem"**: How to determine the **correct output for a given input** ? (e.g., when the oracle is **unknown** or unavailable).

- The oracle problem is the main **obstacle** for automated **test generation**. Without an oracle, we cannot create the tests.
  - Often need human input to define oracles.
- There are many different **types of test oracles**.

© Francisco Oliveira Neto - 2018

Types of Oracles

- **Specified** oracles: Oracles are **extracted** from the system with respect to a given **formal specification**. E.g., a model, state machines, formal verification of the code, etc.
  - Not scalable and depend on formalism chosen.

- **Derived** oracles: Oracles are **obtained** by parsing information from **existing artefacts**. E.g., test results from previous version of the system to build regression suites. Regression tests also rely on derived oracles as previous functionalities should not “break” after changing the software.

© Francisco Oliveira Neto - 2018

Types of Oracles

- **Implicit** Oracles: Oracles are **obtained** from **assumptions** about the system. E.g., they are often related to performance and **robustness** as we expect that the software should not crash and should not be very slow in processing requests such as testing for time-out or buffer overflows.

- **Human** Oracles: Oracles are define by an **expert's opinion or knowledge** about the System Under Test (SUT). E.g., blocking after X failed login attempts, or typical user mistakes.

© Francisco Oliveira Neto - 2018

Fault distribution

- “Bugs are **not uniformely distributed**”. Some parts of the code will have more or less faults than others.

- **Pareto principle (power law):** 80% of the faults lie in 20% of the source code.
  - We mainly change those components that are exercises more by users, so many faults in other components are not triggered often.
- Therefore, we need different levels and strategies of testing to make sure **satisfactory coverage** of the test.

© Francisco Oliveira Neto - 2018

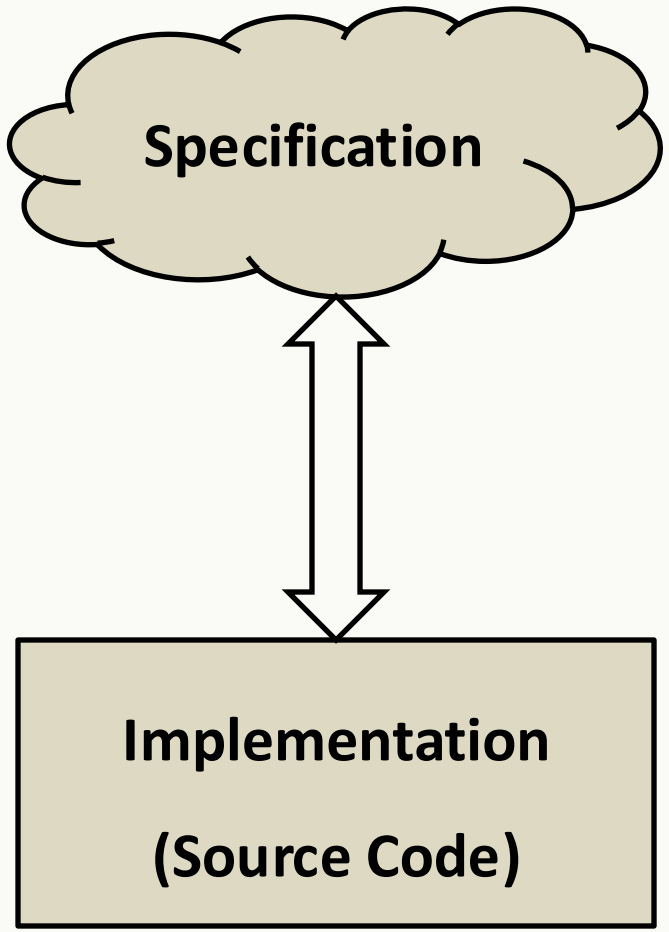
How do we create tests?

- There are different techniques **depending on the level**.
- Some practices applicable to **all testing levels** are:
  - **TDD:** Test-driven development (or Test-first approaches)
  - **Black-box** testing
  - **White-box** testing
  - Gray-box testing

© Francisco Oliveira Neto - 2018

Types of testing

- **Black-box:** Test activities that rely on information from the specification.
  - Mainly input-output relations.
- **White-box:** Test activities that rely on information from the implementation.
  - Knowledge of source code constructs. E.g., access to if statements, loops, method dependencies.
- **Grey-box:** Middle ground between the two above.
  - Access to information about the source code (e.g., types of input, execution traces, etc.)



© Francisco Oliveira Neto - 2018

Exercise: Black and White-box Testing

- Divide the class in two groups.
- Group 1 receives the code of the SUT.
- Group 2 receives the specification of the SUT.
- **Generate 5 test cases.** Then argue and justify:
  - How good are your test cases?
  - What made it easier to design the tests? What made it harder to test it?

© Francisco Oliveira Neto - 2018